

CS690S -AI Alignment - Fall 2026

Sagnik Chatterjee

September 25, 2026

Code: https://github.com/sagnikc395/BCO_assignment

1 Part 1

1.1 Question

Run the following code again

```
python test_gym.py
```

What do you notice happening? What specifically about MountainCar makes the problem difficult for RL?

1.2 Solution

The random policy jitters the car back and forth near the bottom of the valley and never leaves it: every one of the 200 steps returns -1 and the episode ends with a return of -200.0 , with the car still in the basin.

MountainCar is hard for RL because the reward signal carries no information about progress. The engine is weaker than gravity, so the only way to reach the flag is to first drive *away* from it and pump energy into the system over many steps (a long, precisely correlated sequence of actions). A uniformly random policy has essentially zero probability of producing such a sequence within 200 steps, so the agent almost never sees a return other than -200 . Three consequences follow:

- **Exploration / sparse-in-practice reward:** all trajectories look equally bad (-200), so the reward gives no gradient toward the goal. The agent is stuck in a flat region of the return landscape, i.e. a local optimum where nothing it tries locally looks better than anything else.
- **Long credit-assignment horizon:** the action that matters (backing up the left hill) is rewarded only tens of steps later, when the car finally clears the right hill.
- **Counter-intuitive optimal behavior:** a greedy/myopic policy that pushes toward the goal never succeeds; the optimal policy must temporarily move in the wrong direction.

2 Part 2

2.1 Question

You will now learn how to solve the MountainCar task by driving the car yourself. Run

`python mountain_car_play.py`

Use the arrow keys to control the car by accelerating left and right. Note that if you run out of time you get a reward of -200.0 and the car will reset to the bottom of the hill. If you get to the flag in less than 200 steps you can get a higher score. Once you reach the flag the environment will restart at the bottom of the hill.

Keep practicing until you can reliably get out of the valley. You can see your score for each episode output on the command line. Experiment with different strategies: e.g.

1. Going right, then left, then right all the way up the hill
2. Going left, then right all the way up the hill.
3. Left, right, left, right up the hill.

Which strategy do you like best? What is the best score you can get as a human demonstrator?

2.2 Solution

I found strategy 2 (go left first to build up potential energy, then accelerate right all the way up) the easiest and most reliable, because it requires only one direction switch and so is easy to time correctly. Strategy 1 also works but wastes the first few steps; strategy 3 (left, right, left, right) is the hardest to time: if the switches are not in phase with the car's natural oscillation, the pumping cancels out and the episode times out at -200 .

Over roughly 19 practice episodes my returns ranged from -200.0 (timeouts, early on) to a best score of -87.0 , with typical good demonstrations landing in the -140 to -160 range. The -87.0 episode used the “left then right” strategy with a single well-timed switch at the leftmost point of the swing. Table 1 lists every scored episode.

Table 1: Returns for my 18 scored practice episodes in `mountain_car_play.py`, in the order they were played.

episode	1	2	3	4	5	6	7	8	9
return	-165	-155	-88	-200	-176	-146	-169	-151	-157
episode	10	11	12	13	14	15	16	17	18
return	-138	-142	-87	-160	-159	-160	-200	-160	-147

3 Part 3

3.1 Question

Now we will teach a simple behavioral cloning (BC) agent to drive itself out of the valley.

First take a look at `mountain_car_bc.py` and try to get a basic understanding of what is happening. By default this code will collect a single demonstration, then parse the states and actions into tensors for easy input to a PyTorch neural network. It then trains a policy to imitate the demonstrations and evaluates the policy by testing it on different initial states. The command line will output the average, min, and max total rewards.

Try it out by running

```
python mountain_car_bc.py
```

Try to provide a good demonstration. Then watch what the agent has learned. Does it do a good job imitating? Does it ever get stuck? What is the average, min, and max?

3.2 Solution

The demonstration I gave had return -151.0 . BC training drove the cross-entropy loss down smoothly from 1.07 to 0.29 over 100 full-batch iterations, so the policy does fit the demonstration data well.

The imitation is only partly successful. The learned policy reproduces the left-then-right pumping motion, but it is brittle: from initial states that differ from the one the demonstration started in, it drifts off the demonstrated trajectory and then has nothing to fall back on, and in 2 of 6 evaluation episodes it got stuck oscillating in the valley and timed out. Table 2 shows the training loss and Table 3 the per-episode evaluation returns.

Table 2: Part 3 BC training loss (1 demonstration, 100 full-batch iterations), sampled every 10 iterations.

iteration	0	10	20	30	40	50	60	70	80	90	99
BC loss	1.0681	0.8634	0.8158	0.7486	0.6565	0.5703	0.4791	0.4058	0.3511	0.3107	0.2852

Table 3: Part 3 evaluation returns over 6 episodes. Average -177.5 , minimum -200.0 , maximum -146.0 .

evaluation episode	0	1	2	3	4	5
return	-200	-184	-200	-149	-146	-186

So its best episodes are about as good as my demonstration (-146 vs. -151), but it fails outright a third of the time. This is the *compounding error / covariate shift* problem of BC: a single demonstration covers a very narrow slice of the state space, and once a small prediction error pushes the car into a state that never appeared in the data, the policy's predictions there are essentially unconstrained.

4 Part 4

4.1 Question

Let's give more than one demonstration. Run the following to give 5 good demonstrations. If you mess up during one demo, feel free to restart until you give 5 good demos and try to keep to a consistent strategy.

```
python mountain_car_bc.py --num_demos 5
```

Report the average min and max returns. Did it perform any better? Why or why not? Does the agent copy the strategy you used?

Table 4: Part 3 versus Part 4 evaluation statistics over 6 episodes.

	1 demo (Part 3)	5 demos (Part 4)
average return	-177.5	-145.8
minimum return	-200.0	-153.0
maximum return	-146.0	-141.0

Table 5: Part 4 evaluation returns over 6 episodes, all of which reach the flag.

evaluation episode	0	1	2	3	4	5
return	-144	-144	-143	-150	-153	-141

4.2 Solution

With 5 demonstrations (returns $-192, -160, -169, -162, -92$) the learned policy is better and much more *consistent*.

Every one of the 6 evaluation episodes now reaches the flag (no -200 timeouts), and the spread of returns collapsed from 54 to 12.

Why it is better: more demonstrations cover more of the state space, in particular more distinct positions and velocities along the pumping swing and more distinct initial states, so the policy is asked to extrapolate much less at test time and small errors no longer push it into unseen states. The average return also beats the average of my own demonstrations (-155), because the network averages over my inconsistent timing and produces a smoother, more canonical version of the strategy.

Note that the final BC loss is *higher* than in Part 3 (0.52 vs. 0.29): with 5 demos there is more conflicting data (I did not switch direction at exactly the same state every time), so the fit is worse but the generalization is better. The agent does copy my strategy: it plays the left-then-right single-switch policy I demonstrated, not some other solution, which is expected, since BC has no reward signal and can only reproduce the demonstrated behavior mode.

5 Part 5

5.1 Question

What do you think will happen if we give good and bad demonstrations? You will now give two demonstrations. For the first one, just press the right arrow key for the entire episode until it restarts. Then for the second demo, give a good demonstration that quickly gets out of the valley.

```
python mountain_car_bc.py --num_demos 2
```

What does the policy learn? Why might bad demonstrations be a problem? Briefly suggest one potential idea for making BC robust to bad demonstrations, as long as they are a minority of demonstrations.

5.2 Solution

I expected the bad demonstration to pull the policy toward always accelerating right. That is what happened, and it was enough to break the policy entirely. I gave one all-right demonstration (return -200.0 , the car never leaves the valley) and one good demonstration that reached the flag (return

Table 6: Part 4 BC training loss (5 demonstrations, 100 full-batch iterations), sampled every 10 iterations.

iteration	0	10	20	30	40	50	60	70	80	90	99
BC loss	1.1298	0.9885	0.9562	0.9083	0.8679	0.7754	0.6963	0.6235	0.5703	0.5398	0.5230

−116.0). The learned policy never escapes the valley: all six evaluation episodes returned −200.0, so the average, minimum and maximum returns are all −200.0.

Table 7: Part 5 BC training loss (1 bad plus 1 good demonstration), sampled every 10 iterations.

iteration	0	10	20	30	40	50	60	70	80	90	99
BC loss	0.9332	0.5553	0.4392	0.3991	0.3858	0.3751	0.3644	0.3555	0.3474	0.3399	0.3341

The training loss is not what fails here. It drops quickly from 0.93 to about 0.40 within 30 iterations and then flattens into a slow decline to 0.33, i.e. the network fits the mixed dataset about as well as it fit the single clean demonstration in Part 3 (0.29), yet the resulting policy is strictly worse than either demonstrator. The residual floor around 0.33–0.40 is *irreducible label noise*: the two demonstrations visit overlapping states but label them with different actions, so no deterministic policy can fit both. Cross-entropy responds by predicting something close to the state-conditional average of the two behaviors, and because the argmax of that average is dominated by the “push right” action (it is the only action in the bad demo and also appears throughout the second half of the good one), the policy just pushes right and never builds momentum. Averaging two valid-but-different behaviors gives behavior that is worse than either — and low BC loss is no indicator of it, since the loss is measured on the demonstrator’s own states, not on the states the learned policy actually reaches.

Why bad demonstrations are a problem. BC is pure supervised learning on (s, a) pairs and has no notion of demonstration quality: every transition contributes equally to the loss, so a bad demonstrator is imitated just as faithfully as a good one. Worse, in MountainCar the good and bad policies disagree precisely in the states that matter (when to switch direction), so the conflict is not averaged away harmlessly, and it destroys the one behavior that solves the task.

One idea for robustness. Weight demonstrations rather than treating them uniformly: since we can evaluate each demonstration’s return in the environment (or its episode length / whether it reached the flag), fit BC with per-demonstration weights $w_i \propto \exp(R_i/\beta)$, or simply filter to the top- k / above-median demonstrations before training. Because bad demos are assumed to be a minority, the median return is a good threshold and the outliers get (near-)zero weight. If no reward is available at all, an unsupervised variant works similarly: fit BC, then compute each transition’s loss under the fitted policy and down-weight or drop the demonstrations with the highest disagreement (a trimmed-loss / RANSAC-style approach), iterating a few times so that the majority mode wins and the inconsistent minority is discarded.

6 Part 6

6.1 Question

At a conceptual level, describe what changes you would need to make to the BC code to implement BCO(0), as described in the paper we read in class: <https://arxiv.org/pdf/1805.01954.pdf>. Answer this question before starting Part 7.

6.2 Solution

BCO(0) removes the assumption that the demonstrations contain actions: we observe only the state sequences $\{(s_t, s_{t+1})\}$ of the demonstrator and must infer the actions ourselves before we can run BC. Conceptually there are three changes to the BC code, and one full pass through them ($\alpha = 0$ means no post-demonstration iteration):

1. **Pre-demonstration / self-supervised data collection.** Before touching the demonstrations, interact with the environment using a random policy and record (s_t, a_t, s_{t+1}) transitions. Here the actions *are* known because we chose them, so this data is free and needs no human. This is what `collect_random_interaction_data` provides.
2. **Learn an inverse dynamics model.** Train a new network $M_\theta(a \mid s_t, s_{t+1})$ on the random interaction data. Since the action space is discrete (3 actions), this is again a classification problem trained with cross-entropy, structurally the same training loop as BC, but with input (s_t, s_{t+1}) (i.e. 4 input features instead of 2) and the executed action as the label. This model is trained *only* on the self-generated data, never on the demonstrations.
3. **Label the demonstrations, then clone.** Discard the demonstrator's true actions (they are assumed unavailable) and instead run each demonstration transition (s_t, s_{t+1}) through the inverse dynamics model, taking $\hat{a}_t = \arg \max_a M_\theta(a \mid s_t, s_{t+1})$. Feed the resulting $(s_t, \hat{a}_t)$ dataset into the existing `train_policy` BC routine and evaluate as before.

The policy network, the BC loss, and the evaluation loop stay exactly the same; what is added is a second network plus its own data-collection and training stage, and the action labels become *predicted* rather than ground truth. The key practical risk is a distribution mismatch: the inverse dynamics model is trained on states a random policy visits (near the bottom of the valley) but is applied to states the expert visits (high up on both hills), so its accuracy on the demonstration data can be much worse than its training accuracy. (BCO(α) fixes this by alternating: roll out the current learned policy, add those transitions to the inverse-dynamics training set, retrain, and re-label. With $\alpha = 0$ we do a single pass.)

7 Part 7

7.1 Question

Implement and test BCO(0) by modifying the starter code in `mountain_car_bco.py`. This starter code is identical to the BC code, except: (1) the function `collect_random_interaction_data` has been added to help with learning the inverse dynamics model; (2) a new argument for number of inverse dynamics training iterations has been added; and (3) some comments have been added about where to use the inverse dynamics model instead of ground-truth actions. You may need to experiment with different neural network sizes and training iterations to get the inverse dynamics model to work well. Report how well it works and what you tried to get it to work, including stats on the accuracy of prediction when tested on the demonstration data (training should be done with the random interaction data, not the demonstrations).

7.2 Solution

Implementation. I implemented BCO(0) in `mountain_car_bco.py` along the three stages above: an `InverseDynamicsNetwork` (input $(s_t, s_{t+1} - s_t)$, 3-way output) trained with full-batch Adam and cross-entropy on random interaction data only, then used to relabel the demonstration transitions, which are fed to the unchanged BC routine. The demonstrations are cached in `demos.pkl` so that every configuration is scored on *the same* 734 demonstration transitions (5 demos: 196 accelerate-left, 110 no-op, 428 accelerate-right), and each run appends a row of config + metrics to a CSV. `run_p7.sh` sweeps network size and inverse dynamics training iterations with 5 seeds per configuration and 20 evaluation episodes per run (`results_p7.csv`); a second sweep over the amount and coverage of the random interaction data is in `results_p7_coverage.csv`. Every number below is a mean over the 5 seeds.

Accuracy on the demonstration data. Training accuracy on the random interaction data is 1.000 in every configuration except the smallest network, so it is not a useful diagnostic; the informative number is accuracy on the demonstration transitions, which the model never trains on. In the baseline configuration (64 units, 2 hidden layers, 1000 iterations, 5 random episodes = 1000 transitions) demonstration accuracy is only 0.771, and the resulting policy reaches the flag in 36% of evaluation episodes at an average return of -185.2 .

Table 8: Network size: inverse dynamics accuracy on the 734 demonstration transitions, average return and success rate of the resulting policy. Width varies at 2 hidden layers; depth varies at 64 units per layer. Baseline: width 64, 2 layers.

configuration	demo acc.	avg. return	success rate
width 2	0.529	-185.8	0.31
width 4	0.849	-163.3	0.86
width 8	0.830	-165.7	0.70
width 16	0.806	-176.6	0.51
width 32	0.784	-187.2	0.19
width 64 (baseline)	0.771	-185.2	0.36
width 128	0.790	-175.5	0.49
width 256	0.805	-166.8	0.55
1 layer	0.837	-166.8	0.68
3 layers	0.712	-199.5	0.02
4 layers	0.649	-200.0	0.00

Table 9: Inverse dynamics training iterations (width 64, 2 layers): accuracy on the 734 demonstration transitions and average return.

inverse dynamics iterations	10	25	50	100	250	500	1000	3000
demo accuracy	0.612	0.714	0.752	0.760	0.768	0.771	0.771	0.785
avg. return	-200.0	-187.0	-179.3	-182.3	-182.3	-186.3	-185.2	-187.2

Extra capacity was useless or harmful. Only width 2 genuinely underfits, and it is the only configuration whose *training* accuracy fails as well (0.674); above that, *smaller is better*: width 4 is the best configuration in the whole size sweep (0.849 demonstration accuracy, 86% success), and widths 32–256 are all at or below the 64-unit baseline. Depth is actively bad: a single hidden layer

beats the 2-layer baseline, while 3 and 4 layers collapse to 0.02 and 0.00 success. This suggests that a bigger inverse dynamics model fits the random interaction data more tightly and therefore extrapolates worse to the expert’s states, which is exactly where it has to work. Training longer helps only marginally and saturates after roughly 50 iterations: going from 50 to 3000 iterations moves demonstration accuracy from 0.752 to 0.785 and does not improve return at all, so iterations alone do not close the gap.

Where the errors are. Breaking the baseline’s demonstration accuracy down shows what the missing 23% is:

- *By action:* no-op is predicted at 0.933, accelerate-left at 0.755, and accelerate-right — the most common expert action (428 of 734 transitions) — at only 0.736, because the random policy rarely produces the sustained rightward motion the expert uses to climb.
- *By state region:* a uniformly random policy hardly leaves the bottom of the valley, so the baseline model is trained on positions in roughly $[-0.74, -0.32]$ but applied to demonstration states spanning $[-1.19, 0.50]$, leaving about 403 of the 734 demonstration transitions outside the covered position range. On demonstration states *inside* the covered range accuracy is 0.935; on those outside it, it falls to 0.637.

What I had to do to make the inverse dynamics model work. Two things outside network size and iteration count mattered far more than either of them.

1. **State coverage of the random data.** Holding each (still uniformly random) action for k steps lets the car build momentum and extends the random data’s position range dramatically at the *same* budget of 1000 transitions. With $k = 8$ the range grows to $[-1.11, 0.11]$, out-of-range demonstration transitions drop from 403 to 130, demonstration accuracy rises to 0.924, and the policy reaches the flag in 100% of evaluation episodes at an average return of -140.5 . Simply collecting more uniformly random episodes is far less efficient: 200 episodes (40,000 transitions, $40\times$ the data) reach 0.849 accuracy, which action repeat 8 beats with 1000.
2. **Input representation / normalization.** The raw features are on very different scales: position lives in $[-1.2, 0.6]$, while a single action changes velocity by only ± 0.001 . I feed $(s_t, s_{t+1} - s_t)$, i.e. the state plus the *delta*, and standardize all 4 features using statistics computed from the random interaction data. Normalization is what makes the coverage fix pay off: at action repeat 8, turning it off drops demonstration accuracy from 0.924 to 0.824 and accuracy on out-of-range states from 0.647 to 0.305. At the baseline’s tiny coverage it makes no positive difference (0.771 normalized versus 0.818 unnormalized), which is consistent with the same story: when nearly no expert state is in distribution, how the inputs are scaled hardly matters.

How well BCO(0) works. The baseline configuration is poor: 36% success and an average return of -185.2 . With the coverage fix the method works well: demonstration labelling accuracy 0.924, 20/20 successful evaluation episodes for every seed, average return -140.5 , and the best single configuration/seed reached an average return of -120.9 with a best individual episode of -86 . That compares favorably with BC trained on the *true* actions in Part 4 (-145.8), which is the natural reference point for BCO(0) here, and with my own demonstrations (best -87). Variance across seeds is large throughout (the same configuration can go from 0% to 100% success), which is why every number reported here is a 5-seed mean rather than a single run.

Table 10: Coverage of the random interaction data (width 64, 2 layers, 1000 inverse dynamics iterations). Action repeat k holds each random action for k steps; the demo states span $[-1.19, 0.50]$ for comparison.

random data	#trans.	pos. range	demo acc.	#out of range	avg. return	success
5 episodes (baseline)	1000	$[-0.74, -0.32]$	0.771	403	-185.2	0.36
20 episodes	4000	$[-0.80, -0.26]$	0.837	342	-149.5	0.97
50 episodes	10000	$[-0.82, -0.25]$	0.834	329	-167.7	0.74
200 episodes	40000	$[-0.86, -0.18]$	0.849	287	-156.5	0.89
action repeat 2	1000	$[-0.88, -0.18]$	0.868	283	-153.9	0.74
action repeat 4	1000	$[-0.90, -0.13]$	0.885	267	-154.2	0.88
action repeat 8	996	$[-1.11, 0.11]$	0.924	130	-140.5	1.00
action repeat 16	974	$[-1.20, 0.32]$	0.940	49	-145.8	0.89
action repeat 8, no normalization	996	$[-1.11, 0.11]$	0.824	130	-145.7	0.90

So BCO(0) does solve MountainCar, but its performance is governed almost entirely by whether the self-supervised interaction data covers the states the demonstrator visits, and only weakly (and non-monotonically) by the size of the inverse dynamics network or how long it trains. This is consistent with the paper’s motivation for the post-demonstration iterations of BCO(α), which address exactly this mismatch by adding on-policy transitions from the states the learner actually reaches.